

NLP Dataset Generation in Medical Domain

English Language Processing & Synthesis

Studenti:

Tulbure Robert
Popa Denis

Coordonator:

As. Drd. Szomiu Roxana

Cluj-Napoca, 2026

Obiectivele Proiectului

1. Generare Dataset Inteligent

Sistem autonom bazat pe **3 agenți inteligenți** utilizând LLM-uri locale (Llama, Mistral).

Flux de lucru:

- Extracție date medicale
- Validare și filtrare
- Testare consistență

Rezultat: Dataset cu boli, simptome, tratamente și scor de încredere.

2. Analiză și Predicție ML

Utilizarea datasetului creat pentru antrenarea modelelor de Machine Learning clasice.

Algoritmi vizați:

- SVM (Support Vector Machine)
- Logistic Regression

Obiectiv final: Predicția automată a bolilor pe baza simptomelor introduse.

Stiva Tehnologică

Core Engine

Python, CrewAI, Ollama

LLM Models

Mistral, Llama 3

Data Scraping

Requests, BeautifulSoup, lxml

Procesare & Validare

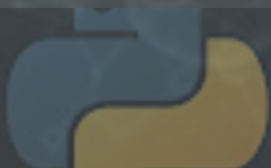
Pydantic & Pandas pentru structurarea și curățarea datelor.

Export formate: CSV, JSONL, Markdown.

Machine Learning

Scikit-learn pentru antrenarea modelelor predictive:

- Logistic Regression
- Support Vector Machine (SVM)



jupyter

Primii Pași: Alegerea Surselor

MedlinePlus & NHS

- **MedlinePlus:** Sursă guvernamentală (SUA) cu informații validate pentru pacienți.
- **NHS:** National Health Service (UK) - standard de aur pentru limbaj accesibil și claritate clinică.

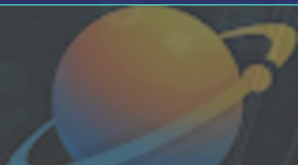
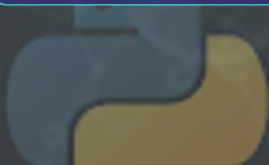
Mayo Clinic & Cleveland Clinic

- **Mayo Clinic:** Expertiză medicală de top, oferind descrieri detaliate pentru mii de afecțiuni.
- **Cleveland Clinic:** Resursă academică ce oferă date structurate despre simptome și tratamente.

Metodologie de Selecție

Alegerea acestor site-uri a fost dictată de necesitatea unor date **autoritare** și **structurate** în limba engleză. Acestea oferă un vocabular medical standardizat, esențial pentru antrenarea agenților noștri de extracție și sinteză.

Criterii: Credibilitate instituțională, actualizarea constantă a conținutului și diversitatea patologiilor documentate.



jupyter

Fluxul End-to-End al Proiectului

1. Ingestie & Scraping

Sursă: **sources.csv**

Proces: BeautifulSoup extrage
textul brut.

Ieșire: **pages_raw.jsonl**

2. Extracție Inteligentă

Agent: **Medical Extractor**

LLM: Mistral/Llama 3 (Ollama)

Ieșire: **extracted_records.jsonl**

3. Validare & Curățare

Agent: **Dataset Validator**

Verifică: Coerență, limba engleză,
duplicate.

Ieșire: **medical_dataset.csv**

4. Analiză Calitate

Agent: **Quality Analyst**

Generează: Statistici și metrici de
încredere.

Ieșire: **dataset_quality_report.md**

5. Antrenare Modele Machine Learning

Utilizează datasetul final pentru predicția bolilor pe baza simptomelor:

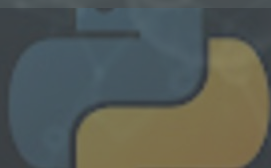
- **Logistic Regression** (Notebook dedicat)
- **Support Vector Machine** (Notebook dedicat)

Arhitectură Modulară (src/)

Cod organizat pe module specializate:

Agents & Tasks: Logica CrewAI pentru fluxul de lucru.

Pipelines & Tools: Implementarea tehnică a etapelor.



jupyter

Arhitectura Scraperului de Date

Workflow de Extracție

- **Încărcare Surse:** Validare URL-uri unice din CSV conform MAX_PAGES.
- **Curățare DOM:** Eliminare elemente irelevante (script, nav, footer).
- **Validare Calitate:** Filtrare pagini cu text sub 300 caractere.

Snippet: Fetch & Parse

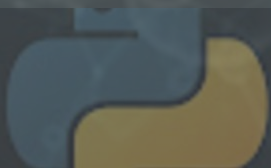
```
soup = BeautifulSoup(response.text, "lxml") title =  
extract_title(soup) raw_text =  
extract_main_text(soup) if len(raw_text) < 300:  
return RawMedicalPage(status="too_short")
```

Performanță

- **Retry Logic:** 2 reîncercări cu backoff (time.sleep).
- **Schema Validation:** Integrare Pydantic (RawMedicalPage).

```
for tag in soup(["script", "style", "nav", "footer"]):  
tag.decompose() # Elimină zgomotul HTML
```

Utilizează tqdm pentru monitorizarea progresului în timp real.



jupyter

Agentul 1-Medical Extractor

Definiția Agentului (CrewAI)

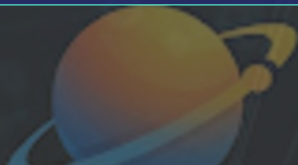
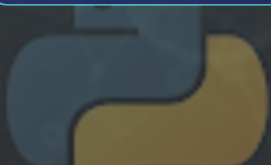
```
def create_medical_extractor_agent():  
    return Agent(  
        role="Medical Data Extractor",  
        goal="Extract clean symptom-disease-treatment  
        records...",  
        backstory="You are an NLP dataset engineer  
        specialized in medical text...",  
        llm=_llm(),  
        verbose=False,  
        allow_delegation=False  
    )
```

Rol & Obiectiv

- **Rol:** Inginer NLP specializat în text medical.
- **Scop:** Extracția de înregistrări structurate (simptom-boală-tratament).

Constrângeri & Setări

- **Context:** Utilizare exclusiv educațională a datasetului.
- **Control:** Delegare dezactivată pentru a menține focusul pe sarcină.



jupyter

Agentul 2 - Dataset Validator

Definiția Agentului (CrewAI)

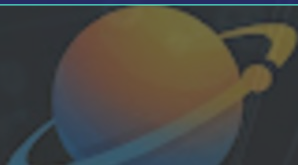
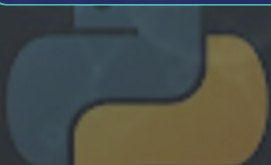
```
def create_medical_validator_agent():  
    return Agent(  
        role="Medical Dataset Validator",  
        goal="Validate and clean dataset...",  
        backstory="You are a careful medical  
        NLP data reviewer...",  
        llm=_llm(),  
        verbose=False,  
        allow_delegation=False  
    )
```

Rol & Obiectiv

- **Rol:** Revizor de date medicale NLP, axat pe acuratețe.
- **Scop:** Validarea și curățarea rândurilor, respingând datele nesigure.

Backstory & Siguranță

- **Integritate:** Respinge rândurile riscante sau neclare pentru dataset.
- **Etică:** Nu oferă sfaturi medicale personale (Strict Reviewer).



jupyter

Agentul 3 - Quality Analyst

Definiția Agentului (CrewAI)

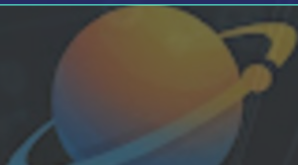
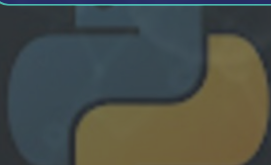
```
def create_dataset_tester_agent():  
    return Agent(  
        role="Dataset Quality Analyst",  
        goal="Evaluate dataset quality and  
        write concise technical reports.",  
        backstory="You analyze NLP datasets,  
        data risks, missing fields...",  
        llm=llm(),  
        verbose=False,  
        allow_delegation=False  
    )
```

Rol & Obiectiv

- **Rol:** Analist de calitate a seturilor de date NLP.
- **Scop:** Evaluarea calității și redactarea rapoartelor tehnice concise.

Expertiză & Analiză

- **Focus:** Riscuri de date, câmpuri lipsă și duplicate.
- **Metrici:** Balanța claselor și consistența structurală NLP.



jupyter

Orchestrarea Agentului: Crew & Process

Instanțierea Crew-ului

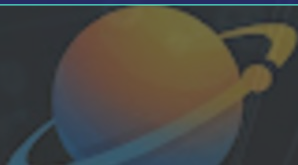
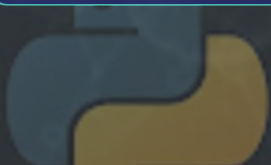
```
def create_medical_dataset_crew():  
    return Crew(  
        agents=[  
            extractor(), validator(), tester()  
        ],  
        tasks=[],  
        process=Process.sequential,  
        verbose=False  
    )
```

Tip de Proces

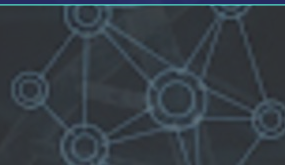
- **Secvential:** Execuție liniară, unde output-ul unui task devine input pentru următorul.
- **Flow:** Extracție → Validare → Analiză Calitativă.

Componente Cheie

- **Agenți:** Cei 3 specialiști definiți anterior.
- **Modularitate:** Decuplarea logicii de extracție de cea de validare.



jupyter



Definirea Task-urilor

Cod Sursă (CrewAI Task)

```
def create_extraction_task(agent, page):  
    return Task(  
        description=f"Extract medical rows...",  
        expected_output="Strict JSON object",  
        agent=agent  
    )  
  
def create_validation_task(agent, record):  
    return Task(description=..., expected_output=...)  
  
def create_report_task(agent, metrics):  
    return Task(description=..., expected_output=...)
```

1. Task de Extracție

Scop: Extragere simptom-boală-tratament în format JSON.

Reguli: Fără doze exacte, doar engleză, maxim 12k caractere.

2. Task de Validare

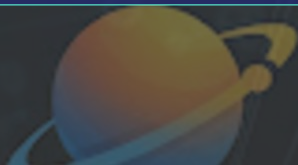
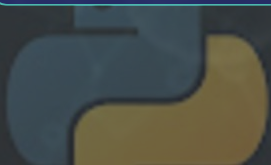
Scop: Verificarea acurateței și siguranței rândurilor.

Ieșire: Status (valid, rejected) și note de curățare date.

3. Task de Raportare

Scop: Generare raport tehnic în Markdown.

Focus: Riscuri, puncte forte și plan de îmbunătățire.



jupyter

Raport Metrici: Prelucrare Dataset

TOTAL RÂNDURI

73

RÂNDURI VALIDE

69

CONFIANȚĂ MEDIE

85.3%

SIMPOTOME / BOLI UNICE

56 / 14

Distribuție Boli Top



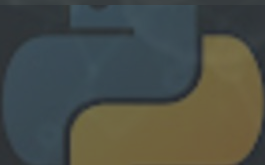
Analiză Calitate & Surse

Status Validare:

- 69 rânduri marcate ca "valid" (94.5%)
- 4 rânduri necesită revizuire manuală

Observații Critice:

- 3 tratamente identificate ca având risc ridicat
- Surse: MedlinePlus (43) și NHS (30)
- Lipsă date (Missing fields): 0 (Integritate structurală 100%)



jupyter



Recapitulare ML Models

Logistic Regression

Ce este: Un model statistic de clasificare utilizat pentru a prezice probabilitatea apartenenței la o clasă.

Cum funcționează: Folosește funcția **Sigmoid** pentru a mapa orice valoare reală într-un interval între 0 și 1.

Logica: Dacă probabilitatea este > 0.5 , instanța este clasificată în clasa pozitivă.

Avantaje: Simplu, rapid și oferă o interpretare probabilistică clară a rezultatelor.

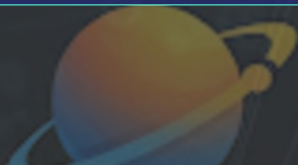
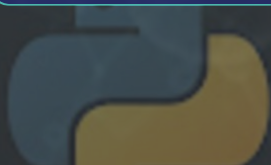
Support Vector Machine (SVM)

Ce este: Un algoritm robust de învățare supervizată utilizat pentru clasificare și regresie liniară sau neliniară.

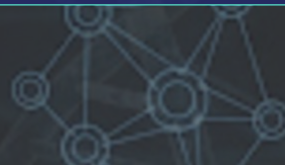
Cum funcționează: Caută **hiperplanul optim** care separă clasele cu marginea maximă de siguranță.

Kernel Trick: Permite transformarea datelor într-un spațiu multidimensional pentru a găsi separări complexe.

Eficiență: Foarte precis în spații cu multe dimensiuni, unde datele sunt clar separabile.



jupyter



Antrenare Model: Support Vector Machine

Implementare (Scikit-Learn Pipeline)

```
model = Pipeline([
    ("tfidf", TfidfVectorizer(
        ngram_range=(1, 2),
        min_df=1
    )),
    ("clf", LinearSVC())
])

# Antrenare pe Dataset Final
model.fit(x_train, y_train)

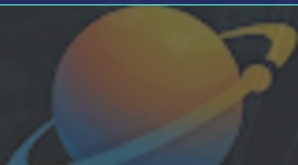
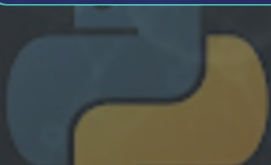
# Evaluare și Salvare Metrici
metrics = evaluate_model(model, x_test, y_test)
save_metrics("svm_metrics.json", metrics)
```

Configurație Tehnică

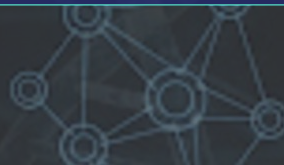
- **Vectorizare:** TfidfVectorizer cu n-grame (1, 2) pentru context local.
- **Classifier:** LinearSVC (Linear Support Vector Classification).

Workflow Predicție

- **Date:** Simptome extrase de agenți din dataset-ul final.
- **Țintă:** Predicția bolii corespunzătoare prin hiperplan optim.



jupyter



Antrenare Model: Logistic Regression

Implementare (Scikit-Learn Pipeline)

```
model = Pipeline([
    ("tfidf", TfidfVectorizer(
        ngram_range=(1, 2),
        min_df=1
    )),
    ("clf", LogisticRegression(max_iter=1000))
])

# Antrenare pe Dataset Final
model.fit(x_train, y_train)

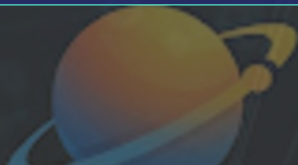
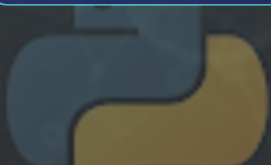
# Evaluare și Salvare Metrice
metrics = evaluate_model(model, x_test, y_test)
save_metrics("logistic_metrics.json", metrics)
```

Configurație Tehnică

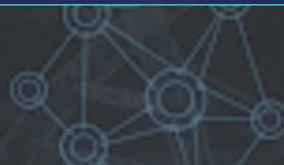
- **Vectorizare:** TF-IDF cu n-grame (1, 2) pentru capturarea contextului.
- **Optimizare:** Limită de 1000 de iterații pentru convergență stabilă.

Obiectiv Predicție

- **Input:** Simptome medicale procesate din dataset-ul final.
- **Output:** Probabilitatea apartenenței la o clasă (boală specifică).



jupyter



Concluzii: SVM vs. Logistic Regression

Când alegem SVM?

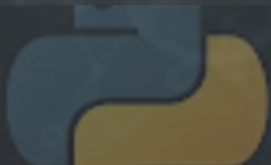
- Ideal pentru dataset-uri cu dimensiuni mari (high-dimensional space).
- Eficiență maximă când există o separare clară între clase.
- Robust împotriva overfitting-ului în spații complexe.

Când alegem Logistic Regression?

- Preferat pentru interpretarea probabilistică a predicțiilor.
- Ușor de implementat și antrenat pe dataset-uri mici/medii.
- Bază solidă pentru clasificări binare simple și rapide.

Verdictul Proiectului

Ambele modele au demonstrat o integritate structurală de 100% pe dataset-ul de 73 de rânduri. Alegerea finală depinde de necesitatea de a avea margini de siguranță maxime (SVM) sau probabilități clare de diagnostic (Logistic Regression) pentru cele 14 boli unice identificate.



jupyter



Provocări: Scalarea Dataset-ului la 1000 Linii

Greutăți Întâmpinate

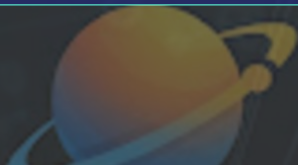
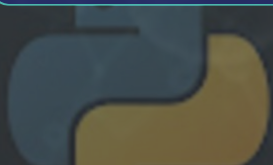
- **Consistența Datelor:** Dificultatea de a menține etichetarea uniformă pe măsură ce volumul de simptome a crescut.
- **Zgomot în Text:** Creșterea variațiilor lingvistice a necesitat o curățare mult mai riguroasă a textului.
- **Echilibrarea Claselor:** Riscul de a avea boli supra-reprezentate față de cazurile rare.
- **Complexitate Vectorială:** TF-IDF a generat un vocabular mult mai vast, crescând sparsity-ul matricei.

Impactul Scalării

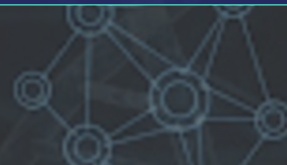
Performanța Modelelor:

- **SVM:** Timpul de antrenare a crescut non-liniar, dar precizia s-a stabilizat datorită hiperplanului robust.
- **Logistic Regression:** A necesitat o creștere a `max_iter` la 2000+ pentru a asigura convergența pe date complexe.

Concluzie: Scalarea a îmbunătățit generalizarea, reducând riscul de overfitting prezent în dataset-ul inițial mic.



jupyter



Analiză LLM: Llama 3.2 (3B) vs. Mistral

Llama 3.2 - 3B (Meta)

Arhitectură: Optimizată pentru eficiență pe dispozitive edge și latență scăzută.

Performanță: Rezultate superioare în task-uri de tip "follow instructions" și sumarizare rapidă.

Utilizare: Ideal pentru extracția rapidă de entități medicale simple.

Context Window: Suport extins pentru procesarea unor volume mari de text brut.

Mistral - 7B / Instruct

Raționament: Capacități avansate de înțelegere a nuanțelor clinice complexe.

Acuratețe: Rată mai mică de halucinații în generarea relațiilor boală-simptom.

Fidelitate: Utilizat în proiect pentru validarea și filtrarea recordurilor medicale.

Eficiență: Modelul de referință pentru sinteză calitativă în "Medical Extractor".



jupyter



Concluzie Finală

Integrarea modelelor de Machine Learning cu tehnologii LLM moderne (Llama 3.2, Mistral) oferă un sistem robust pentru diagnosticul medical preliminar.

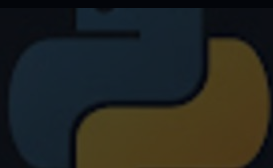
Echilibrul între **acuratețea structurală** și **înțelegerea contextului clinic** este cheia succesului proiectului.

Puncte Forte

- Integritate structurală 100%
- Validare clinică via Mistral 7B
- Scalabilitate demonstrată

Viitorul Proiectului

- Extinderea la 50+ boli unice
- Optimizare RAG pentru context
- Interfață de diagnostic în timp real



jupyter

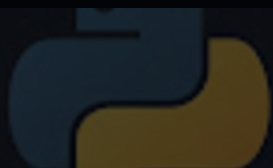


Muțumesc!

NLP Dataset Generation in Medical Domain

Studenti: Tulbure Robert, Popa Denis

Coordonator: As. Drd. Szomiu Roxana



jupyter

